Entornos de Desarrollo

UT 02: Control de versiones.

ÍNDICE

1.- Control de versiones....4
2.- Estructura de las herramientas de control de versiones....5
3.- Repositorio....7
4.- Instalación....9
4.1 Configuración inicial....10
4.2 Creación de un repositorio....10
4.3 Fases de un repositorio....11
4.4 Clonado de un repositorio....12
4.5 Concepto branch....13
4.6 Satus....17
4.6 Log....18
4.7 push/pull/fentch....19
5.- Mecanismos de actualización....22
5.1 add....22
5.2 commit....23
5.3 merge....24
5.4 Git Rebase....25
5.5 Merge Conflicts....26
5.6 Git Merge vs Git Rebase....27
6.- Uso básico de un entorno de desarrollo: Edición de programas. Generación de ejecutables....27
7.- Ventajas e inconvenientes del desarrollo de aplicaciones con entornos de desarrollo con respecto al uso de tecnologías tradicionales....28

1.- Control de versiones.

Antiguamente cuando llegabas a una empresa, normalmente tenías que mantener la página web. Estaba en un pequeño servidor, el flujo era sencillo, hacías una modificación y subías la copia al servidor por ftp.

Si de repente una nueva persona empezaba a trabajar en la empresa, y los dos modifican la aplicación a la vez, cuando cada uno suba su copia, elimina lo que el otro había hecho.

Aquí es donde entra en juego un sistema de control de versiones, un sistema externo que controla los cambios que hace cada persona dentro del equipo como, por ejemplo, Git.

Cuando trabajamos con Git podemos tener acceso a ficheros creados en el inicio de los tiempos de la aplicación. Con Git podemos volver a versiones anteriores del proyecto y también tenemos un historial para mostrar quién o cuando ha hecho cambios en la aplicación.

Para Git el término proyecto no existe como tal, se conoce como un repositorio. La forma de trabajar de Git es con un repositorio distribuido, en el cual, todo nuestro equipo de trabajo tiene una copia tal cual, del repositorio, también hay un repositorio central donde nuestro equipo sube los cambios y a su vez recibe los cambios de los demás compañeros.

Vamos viendo que el mundo empresarial y/o laboral tiene una gran utilidad, pero ¿qué es un control de versiones, y por qué debería importarte? Un control de versiones es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, de modo que puedas recuperar versiones específicas más adelante.

Si eres diseñador gráfico o de web y quieres mantener cada versión de una imagen o diseño (es algo que sin duda vas a querer), usar un sistema de control de versiones (VCS por sus siglas en inglés) es una decisión muy acertada. Dicho sistema te permite regresar a versiones anteriores de tus archivos, regresar a una versión anterior del proyecto completo, comparar cambios a lo largo del tiempo, ver quién modificó por última vez algo que pueda estar causando problemas, ver quién introdujo un problema y cuándo, y mucho más.

Usar un VCS también significa generalmente que si arruinas o pierdes archivos, será posible recuperarlos fácilmente.

> *“You can do a lot of things with git, and many of the rules of what you *should* do are not so much technical limitations but are about what works well when working together with other people. So git is a very powerful set of tools.”*

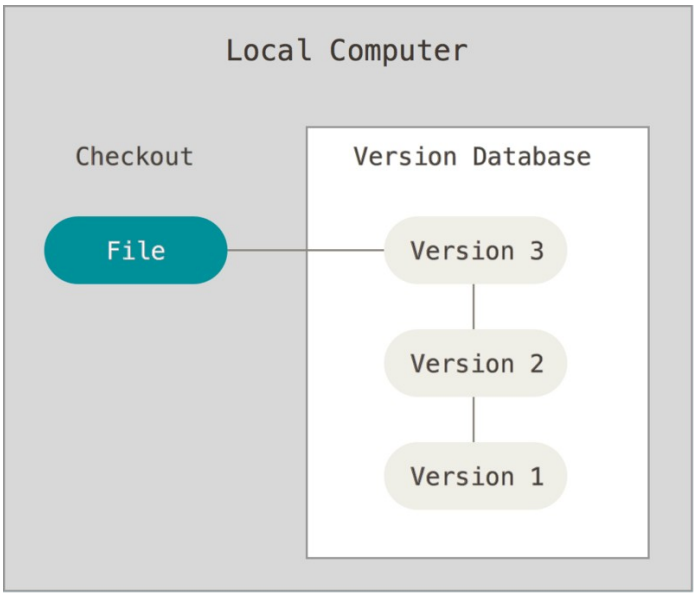
- Linus Torvalds

2.- Estructura de las herramientas de control de versiones.

A la hora de hablar de estructuras de los controles de versiones, nos centraremos en la manera en la que se hospedan cada una de ellas. Esta peculiaridad nos hará tener una estructura u otra en nuestra herramienta de control de versiones

VCS Local

Un método de control de versiones, usado por muchas personas, es copiar los archivos a otro directorio (quizás indicando la fecha y hora en que lo hicieron, si son ingeniosos). Este método es muy común porque es muy sencillo, pero también es tremendamente propenso a errores. Es fácil olvidar en qué directorio te encuentras y guardar accidentalmente en el archivo equivocado o sobrescribir archivos que no querías.

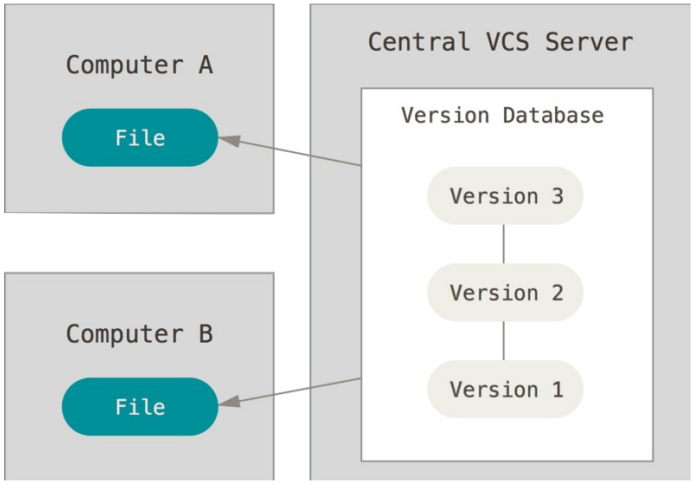


Para afrontar este problema los programadores desarrollaron hace tiempo VCS locales que contenían una simple base de datos, en la que se llevaba el registro de todos los cambios realizados a los archivos.

Una de las herramientas de control de versiones más popular fue un sistema llamado RCS, que todavía podemos encontrar en muchas de las computadoras actuales. Incluso el famoso sistema operativo Mac OS X incluye el comando rcs cuando instalas las herramientas de desarrollo. Esta herramienta funciona guardando conjuntos de parches (es decir, las diferencias entre archivos) en un formato especial en disco, y es capaz de recrear cómo era un archivo en cualquier momento a partir de dichos parches.

VCS Centralizados

El siguiente gran problema con el que se encuentran las personas es que necesitan colaborar con desarrolladores en otros sistemas. Los sistemas de Control de Versiones Centralizados (CVCS por sus siglas en inglés) fueron desarrollados para solucionar este problema. Estos sistemas, como CVS, Subversion y Perforce, tienen un único servidor que contiene todos los



archivos versionados y varios clientes que descargan los archivos desde ese lugar central. Este ha sido el estándar para el control de versiones por muchos años.

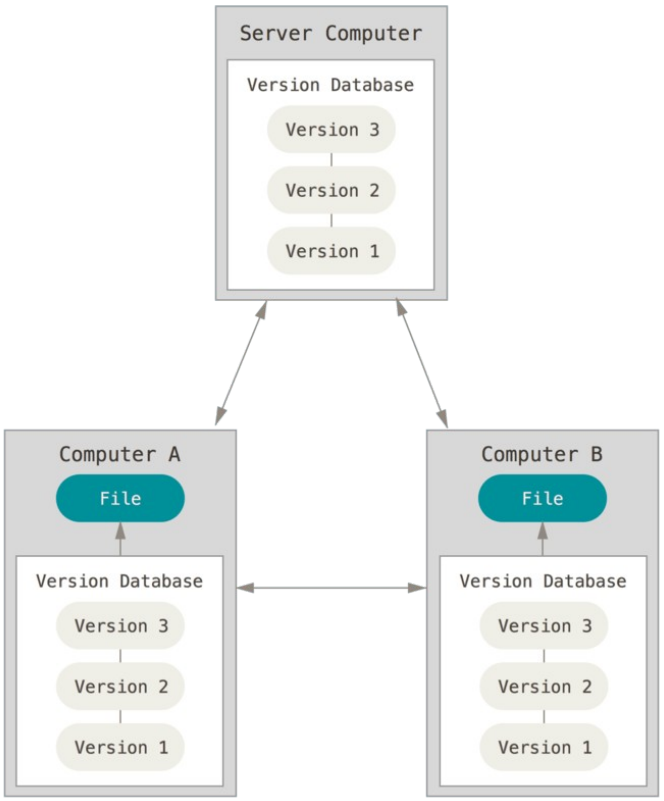
Esta configuración ofrece muchas ventajas, especialmente frente a VCS locales.

Por ejemplo, todas las personas saben hasta cierto punto en qué están trabajando los otros colaboradores del proyecto. Los administradores tienen control detallado sobre qué puede hacer cada usuario, y es mucho más fácil administrar un CVCS que tener que lidiar con bases de datos locales en cada cliente.

Sin embargo, esta configuración también tiene serias desventajas. La más obvia es el punto único de fallo que representa el servidor centralizado. Si ese servidor se cae durante una hora, entonces durante esa hora nadie podrá colaborar o guardar cambios en archivos en los que hayan estado trabajando. Si el disco duro en el que se encuentra la base de datos central se corrompe, y no se han realizado copias de seguridad adecuadamente, se perderá toda la información del proyecto, con excepción de las copias instantáneas que las personas tengan en sus máquinas locales. Los VCS locales sufren de este mismo problema: Cuando tienes toda la historia del proyecto en un mismo lugar, te arriesgas a perderlo todo.

CVS Distribuidos

Los sistemas de Control de Versiones Distribuidos (DVCS por sus siglas en inglés) ofrecen soluciones para los problemas que han sido mencionados. En un DVCS (como Git, Mercurial, Bazaar o Darcs), los clientes no solo descargan la última copia instantánea de los archivos, sino que se replica completamente el repositorio. De esta manera, si un servidor deja de funcionar y estos sistemas estaban colaborando a través de él, cualquiera de los repositorios disponibles en los clientes puede ser copiado al servidor con el fin de restaurarlo. Cada clon es realmente una copia completa de todos los datos.



Además, muchos de estos sistemas se encargan de manejar numerosos repositorios remotos con los cuales pueden trabajar, de tal forma que puedes colaborar simultáneamente con diferentes grupos de personas en distintas maneras dentro del

mismo proyecto. Esto permite establecer varios flujos de trabajo que no son posibles en sistemas centralizados, como pueden ser los modelos jerárquicos.

3.- Repositorio.

A la par de las posibles configuraciones de los sistemas de control de versiones, se definen los repositorios. En Git un repositorio es un proyecto de software (APP, web, documento, etc..) controlado por este sistema de control de versiones. Físicamente no es más que una carpeta del sistema de archivos con código fuente de una aplicación. Esa carpeta se mantiene bajo el seguimiento de Git, de modo que, si se realizan cambios en sus archivos, se mantienen controlados por el sistema.

Todo repositorio tiene dentro una carpeta oculta llamada .git que es donde Git mantiene sus archivos para realizar su propia tarea de control del software. Si por cualquier motivo se borra esa carpeta, dejaría de ser un repositorio y se perdería todo el control que Git mantiene sobre ella. Además, al borrar la carpeta .git también se pendería todo el control de cambios realizado por Git hasta la fecha sobre ese repositorio.

Repositorios distribuidos

Git tiene la característica de ser un sistema de control de versiones con repositorios distribuidos. Esto quiere decir que cada uno de los integrantes de un proyecto tiene una copia exacta del repositorio y no solamente una copia de los archivos con los que haya trabajado. En otras palabras, cuando varios desarrolladores trabajan en un proyecto gestionado por Git, cada uno de ellos tiene en su ordenador un clon de todo el proyecto. Una copia de cada uno de los archivos que forman parte del proyecto.

Cada copia del repositorio o clon contendrá todos los archivos también y realizaría seguimiento igualmente, independientemente de si esa copia está en el mismo ordenador, en un servidor remoto, tipo GitHub, o en el ordenador de otro desarrollador.

Esta situación es muy útil, porque cada desarrollador puede trabajar de manera autónoma, al contar con una copia completa de todo el software que se está desarrollando, sin depender de estar situado en la oficina o de una conexión a Internet. Otros tipos de sistemas de control de versiones que son centralizados no permiten esa posibilidad de trabajo autónomo, ya que el desarrollador requiere estar conectado al repositorio central para poder obtener archivos del proyecto.

Repositorios remotos

Habitualmente se utilizan repositorios remotos, alojados en un servidor de Internet, para que todos los desarrolladores envíen cambios y sincronicen su repositorio local con el repositorio remoto. De este modo se facilita que todas las personas que trabajan en el proyecto puedan colaborar. Proporcionar un repositorio remoto es una de las funciones de GitHub y otros servicios de hosting de repositorios Git como Gitlab o Bitbucket.
Lo dicho anteriormente del posible borrado de la carpeta .git, perdiendo toda la gestión del repositorio local y el control de cambios realizado, o incluso el borrado completo de un proyecto en un ordenador de desarrollo, no sería tanto problema si ese repositorio está subido a GitHub. El hosting de repositorios remotos nos asegura que los cambios se mantendrían salvaguardados. En cualquier momento podríamos volver a descargar el repositorio, mediante la operación de clonando.

Es por ello que debemos tener muy claro cada uno del funcionamiento de los repositorios tanto a la hora de trabajar como a la hora de crear un nuevo proyecto. El uso de uno u otro es fundamental en el mundo laboral y es por ello que se deben conocer y utilizar cada uno ellos en las situaciones en las que mayor beneficio se obtenga del mismo

4.- Instalación

La instalación de Git es muy sencilla, de hecho, en la mayoría de las ocasiones se trata simplemente de instalarlo con el típico programa de instalación de software.

Importante mencionar que el instalador de Git se debe descargar de la página oficial de Git. A partir de ahí, el proceso depende del sistema operativo con el que estemos trabajando. Para los sistemas más importantes es el siguiente.

Instalar Git en Linux

En el caso de Linux, la mayoría de distribuciones ya tienen Git instalado de manera predeterminada. No tenemos que hacer nada en concreto. En Linux o cualquier otro sistema donde quieras saber si Git ya está instalado es lanzar este comando:

```
git --version
```

Si está instalado tendrás un mensaje con la versión disponible en tu sistema.

Instalar Git en Windows

En Windows Git se instala por medio de un instalador. No hay más que seguir el asistente y tendremos ya disponible el comando "git" en nuestro terminal. Los usuarios de Windows durante la instalación de Git tienen la opción de instalar Git Bash, que es un programa de terminal de línea de comandos muy recomendable.

Git Bash es un terminal basado en Bash que por tanto tiene un juego de comandos como el de Linux. Es interesante instalarlo si te dedicas al desarrollo, ya que es mucho más adecuado que el terminal que viene instalado por defecto en Windows.

Instalación de Git en Mac

El instalador para Mac también se realiza mediante la descarga proporcionada en el sitio oficial. Aunque algunas versiones recientes de MacOS ya tienen Git disponible de casa.

Además, para la instalación en Mac existen otra alternativa que merece la pena comentar.

Instalar Git con Homebrew Esto es un instalador de paquetes para MacOS muy útil, más tarde o temprano lo necesitarás para instalar muchas cosas. Una vez instalado Homebrew lanzas el comando

```
brew install git
```

4.1 Configuración inicial

En el momento que acabamos de instalar Git es necesario realizar una primera configuración en la que informaremos a Git sobre quién somos y cuál es nuestro email, que usará para firmar las confirmaciones de código que realicemos.

Para hacer esta configuración inicial debemos lanzar los comandos siguientes:

```
git config --global user.name "Tu nombre"
git config --global user.email "tu_email@example.com"
```

4.2 Creación de un repositorio

Para crear un repositorio Git desde cero, debemos partir de una carpeta vacía. Es por ello que mediante el terminal llegaremos al directorio que queramos. Estando situados en ella con el terminal de línea de comandos, se tiene que lanzar el comando:

```
git init
```

Eso hará que se cree dentro la subcarpeta .git y que la carpeta donde nos encontramos se convierta por tanto en un repositorio Git, pasando a estar controlada por el sistema de control de versiones.

Si deseas empezar a controlar versiones de archivos existentes (a diferencia de un directorio vacío), probablemente deberías comenzar el seguimiento de esos archivos y hacer una confirmación inicial. Puedes conseguirlo con unos pocos comandos git add para especificar qué archivos quieres controlar, seguidos de un git commit para confirmar los cambios:

```
git add *
git add LICENSE
git commit -m 'initial project version'
```

Los repositorios públicos de GitHub se suelen utilizar para compartir software de código abierto. Para que tu repositorio sea verdaderamente de código abierto, tendrás que generarle una licencia. De este modo, las demás personas podrán usar, modificar y distribuir el software con libertad. Si quieres profundizar sobre este tema, está la web choosealicense.com, pero ten en cuenta si subes tu repositorio a GitHub sin licencia otros usuarios pueden ver el repositorio y bifurcarlo (fork).

La mayoría de lis usuarios personas colocan el texto de su licencia en un archivo denominado LICENSE.txt (o bien LICENSE.md o LICENSE.rst) en la raíz del repositorio. Como buena práctica os animo a incluir el fichero de licencia en vuestros proyectos.

4.3 Fases de un repositorio

Ahora que sabemos cómo se crea un repositorio, es importante conocer las fases por las que pasa todo repositorio. En concreto pasa por tres fases:

- Working Directory
- Staging Area
- Git Repository

Cada fase se identifica con un estado:

- Modified
- Staged
- Commited

Fase 1: Working Directory

Aquí es donde podemos hacer cualquier cambio y no afectar nuestro repositorio en lo absoluto. En cuanto modificamos algo de nuestro código, éste tiene status de modified. Si ejecutamos el comando git status, nos mostrará qué archivos han sido modificados (o creados).

Una vez que hemos hecho los cambios necesarios, pasamos nuestros archivos al "staging area" con el comando git add. Cuando pasas el código de Working Directory a Staging Area, cambias el estado del código de modified a staged.

Fase 2: "Staging Area"

Aquí es donde le podemos dar nombre a nuestra nueva versión. Y crear una "copia" de cómo quedaría nuestro repositorio en producción. Para pasar nuestro código de staging area al Git Repository (todavía no se publica el código en Github), escribimos el siguiente comando:

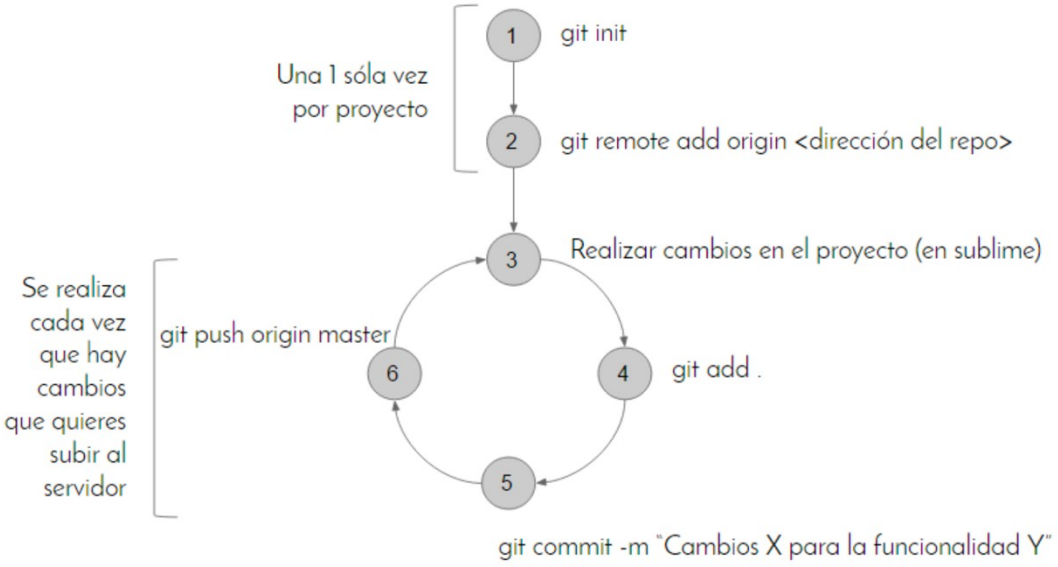
```
git commit -m 'Mensaje de la confirmación'
```

Nota que cuando haces el commit el código pasa de estado staged a commited.

Fase 3. "Git repository"

Una vez que el código está confirmado, committed, ya está listo para sincronizarse con el servidor de Git. Para hacer esto, utilizamos git push.

Si estás trabajando en un repositorio con varias personas en algún momento vas a querer descargar las cosas que tus compañeros agreguen o modifiquen. Para eso ejecutamos git pull origin main.



4.4 Clonado de un repositorio

Cuando iniciamos un proyecto nosotros mismos tenemos que comenzar por inicializar el repositorio con "git init", sin embargo, cuando queremos trabajar sobre un proyecto que ya ha sido iniciado por otras personas, o por nosotros mismos, pero lo queremos obtener desde otro ordenador o servidor, la operación que tendremos que realizar es el clonado.

Por medio del clonado se accede a un repositorio remoto y se obtiene una copia del mismo, que se instala en una carpeta de nuestro ordenador. Una vez clonado podemos comenzar a trabajar con el repositorio realizando modificaciones en su código y confirmaciones (commits) en local.

El comando para hacer el clonado es "clone", seguido de la URL del repositorio que se desea clonar:

```
git clone https://github.com/usuario/nombre_repo.git
```

La URL específica del repositorio la encontrarás en la página del hosting remoto que estés usando, como GitHub o similares.

Una vez hecho el clon se habrá creado en tu ordenador, en la carpeta donde estabas situado con el terminal, un subdirectorio con el mismo nombre que el repositorio. Dentro de ese subdirectorio está el propio clon que acabamos de realizar, con todos los archivos del proyecto.

También puedes clonar un repositorio en otra carpeta que no tiene por qué tener necesariamente el mismo nombre del repositorio. Para ello necesitas situarte en una carpeta completamente vacía, sin ningún archivo, que habrás creado con el nombre que quieras. Luego lanzas el comando de clonado, pero agregando un punto al final.

```
git clone https://github.com/usuario/nombre_repo.git .
```

Con eso le estás diciendo a Git que el clonado debe realizarse en la carpeta actual (el punto del final significa el directorio actual como carpeta de destino del clon). Pero es importante que la carpeta actual esté completamente vacía, ya que en caso contrario daría un error.

4.5 Concepto branch

Antes de entrar en el estado de nuestro repositorio, hay que introducir el concepto de branch o rama, que es clave a la hora de entender el funcionamiento de Git.

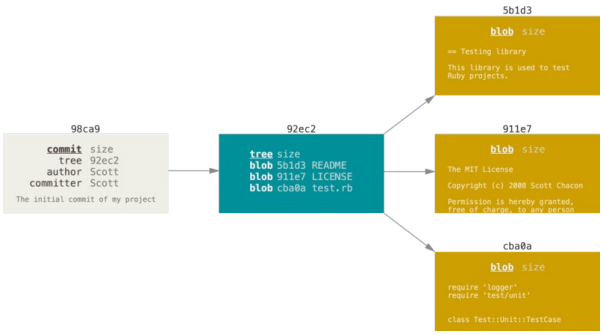
Para entender realmente cómo ramifica Git, previamente hemos de examinar la forma en que almacena sus datos. Git no los almacena de forma incremental (guardando solo diferencias), sino que los almacena como una serie de instantáneas (copias puntuales de los archivos completos, tal y como se encuentran en ese momento).

En cada confirmación de cambios (commit), Git almacena una instantánea de tu trabajo preparado. Dicha instantánea contiene además unos metadatos con el autor y el mensaje explicativo, y uno o varios apuntadores a las confirmaciones (commit) que sean padres directos de esta (un padre en los casos de confirmación normal, y múltiples padres en los casos de estar confirmando una fusión (merge) de dos o más ramas).

Para ilustrar esto, vamos a suponer, por ejemplo, que tienes una carpeta con tres archivos, que preparas (stage) todos ellos y los confirmas (commit). Al preparar los archivos, Git realiza una suma de control de cada uno de ellos almacena una copia de cada uno en el repositorio (estas copias se denominan "blobs"), y guarda cada suma de control en el área de preparación (staging area):

```
git add README test.rb LICENSE
git commit -m 'initial commit of my project'
```

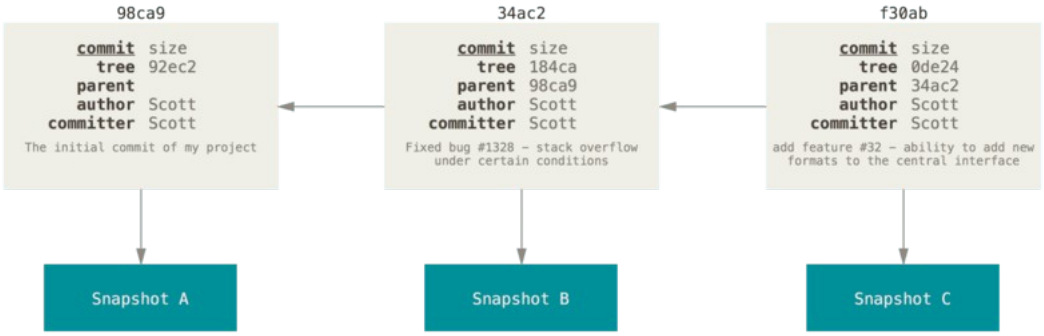
Cuando creas una confirmación con el comando git commit, Git realiza sumas de control de cada subdirectorio (en el ejemplo, solamente tenemos el directorio principal del proyecto), y las guarda como objetos árbol en el repositorio Git. Después, Git crea



un objeto de confirmación con los metadatos pertinentes y un apuntador al objeto árbol raíz del proyecto.

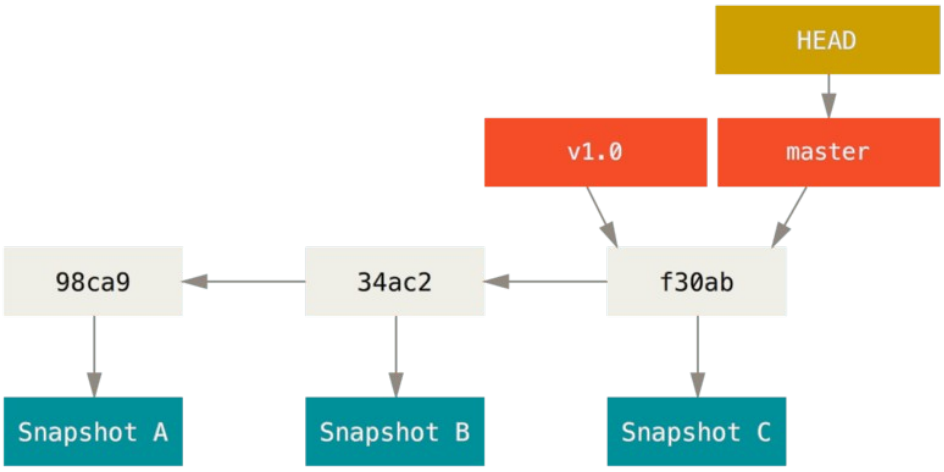
En este momento, el repositorio de Git contendrá cinco objetos: un "blob" para cada uno de los tres archivos, un árbol con la lista de contenidos del directorio (más sus respectivas relaciones con los "blobs"), y una confirmación de cambios (commit) apuntando a la raíz de ese árbol y conteniendo el resto de metadatos pertinentes.

Si haces más cambios y vuelves a confirmar, la siguiente confirmación guardará un apuntador a su confirmación precedente.



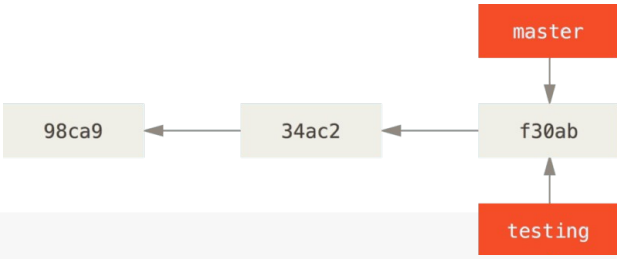
Una rama Git es simplemente un apuntador móvil apuntando a una de esas confirmaciones. La rama por defecto de Git es la rama master. Con la primera confirmación de cambios que realicemos, se creará esta rama principal master apuntando a dicha confirmación. En cada confirmación de cambios que realicemos, la rama irá avanzando automáticamente.

Nota	La rama "master" en Git, no es una rama especial. Es como cualquier otra rama. La única razón por la cual aparece en casi todos los repositorios es porque es la que crea por defecto el comando git init y la gente no se molesta en cambiarle el nombre.



Crear una Rama Nueva

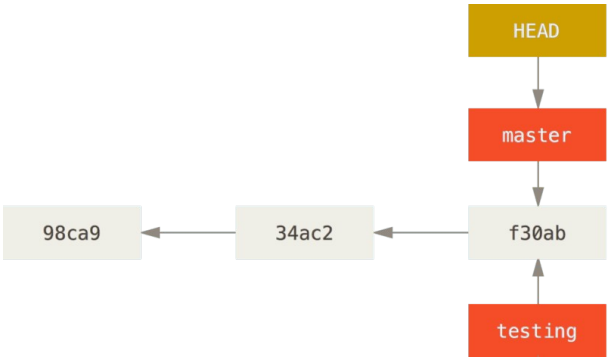
¿Qué sucede cuando creas una nueva rama? Simplemente se crea un nuevo apuntador para que lo puedas mover libremente. Por ejemplo, supongamos que quieres crear una rama nueva denominada "testing". Para ello, usarás el comando git branch:




```
git branch testing
```

Esto creará un nuevo apuntador apuntando a la misma confirmación donde estés actualmente.

¿Cómo sabe Git en qué rama estás en este momento? Mediante un apuntador especial denominado HEAD. Aunque es preciso comentar que este HEAD es totalmente distinto al concepto de HEAD en otros sistemas de control de cambios como Subversion o CVS. En Git, es simplemente el apuntador a la rama local en la que tú estés en ese momento, en este caso la rama master; pues el comando git branch solamente crea una nueva rama, pero no salta a dicha rama.

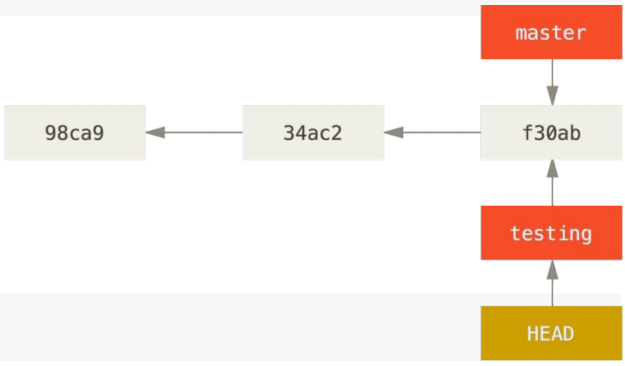


Esto puedes verlo fácilmente al ejecutar el comando git log para que te muestre a dónde apunta cada rama. Esta opción se llama --decorate.

```
git log --oneline --decorate
f30ab (HEAD, master, testing) add feature #32 - ability to add new
34ac2 fixed bug #1328 - stack overflow under certain conditions
98ca9 initial commit of my project
```

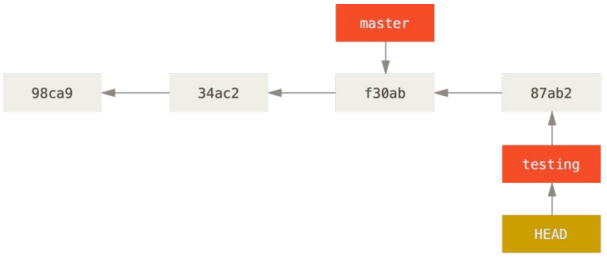
Puedes ver que las ramas “master” y “testing” están junto a la confirmación f30ab.

Para saltar de una rama a otra, tienes que utilizar el comando git checkout. Hagamos una prueba, saltando a la rama testing recién creada:




```
git checkout testing
```

Esto mueve el apuntador HEAD a la rama testing.

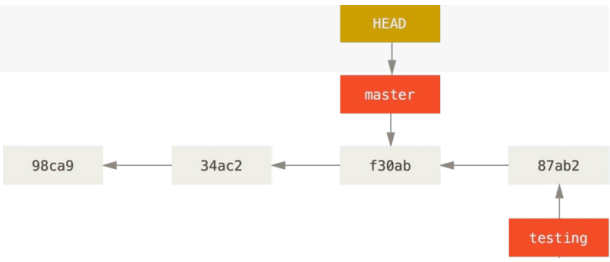


¿Cuál es el significado de todo esto? Bueno, lo veremos tras realizar otra confirmación de cambios:

```
vim test.rb
git commit -a -m 'made a change'
```

Observamos algo interesante: la rama testing avanza, mientras que la rama master permanece en la confirmación donde estaba cuando lanzaste el comando git checkout para saltar. Volvamos ahora a la rama master:

```
git checkout master
```



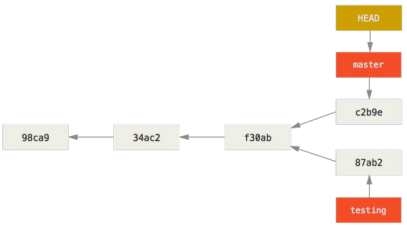
Este comando realiza dos acciones: mueve el apuntador HEAD de nuevo a la rama master, revierte los archivos de tu directorio de trabajo; dejándolos tal y como estaban en la última instantánea confirmada en dicha rama master. Esto supone que los cambios que hagas desde este momento en adelante, divergirán de la antigua versión del proyecto. Básicamente, lo que se está haciendo es rebobinar el trabajo que habías hecho temporalmente en la rama testing; de tal forma que puedas avanzar en otra dirección diferente.

Nota	Saltar entre ramas cambia archivos en tu directorio de trabajo Es importante destacar que cuando saltas a una rama en Git, los archivos de tu directorio de trabajo cambian. Si saltas a una rama antigua, tu directorio de trabajo retrocederá para verse como lo hacía la última vez que confirmaste un cambio en dicha rama. Si Git no puede hacer el cambio limpiamente, no te dejará saltar.

Haz algunos cambios más y confírmalos:

```
vim test.rb
git commit -a -m 'made other changes'
```

Ahora el historial de tu proyecto diverge. Has creado una rama y saltado a ella, has trabajado sobre ella; has vuelto a la rama original, y has trabajado también sobre ella. Los cambios realizados en ambas sesiones de trabajo están aislados en ramas independientes: puedes saltar libremente de una a otra según estimes oportuno. Y todo ello simplemente con tres comandos: git branch, git checkout y git commit.



También puedes ver esto fácilmente utilizando el comando git log. Si ejecutas git log --oneline --decorate --graph --all te mostrará el historial de tus confirmaciones, indicando dónde están los apuntadores de tus ramas y como ha divergido tu historial.

```
git log --oneline --decorate --graph --all
* c2b9e (HEAD, master) made other changes
| * 87ab2 (testing) made a change
|/
* f30ab add feature #32 - ability to add new formats to the
* 34ac2 fixed bug #1328 - stack overflow under certain conditions
* 98ca9 initial commit of my project
```

Debido a que una rama Git es realmente un simple archivo que contiene los 40 caracteres de una suma de control SHA-1, (representando la confirmación de cambios a la que apunta), no cuesta nada el crear y destruir ramas en Git. Crear una nueva rama es tan rápido y simple como escribir 41 bytes en un archivo, (40 caracteres y un retorno de carro).

Esto contrasta fuertemente con los métodos de ramificación usados por otros sistemas de control de versiones, en los que crear una rama nueva supone el copiar todos los archivos del proyecto a un directorio adicional nuevo. Esto puede llevar segundos o incluso minutos, dependiendo del tamaño del proyecto; mientras que en Git el proceso es siempre instantáneo. Además, debido a que se almacenan también los nodos padre para cada confirmación, el encontrar las bases adecuadas para realizar una fusión entre ramas es un proceso automático y generalmente sencillo de realizar. Animando así a los desarrolladores a utilizar ramificaciones frecuentemente.

4.6 Satus

Hay una manera de saber cuál es estado de nuestro de nuestro repositorio es el comando status

```
git status
```

Esto nos da el estado de todos los ficheros que han sido modificados y su estado con el repositorio, si han sido añadidos o no. Además hay dos comandos que nos dan más o menos información sobre esto como son:

De manera simplificada

```
git status -s
```

Información de la rama en la que estamos trabajando

```
git status -s -b
```

4.6 Log

El comando git log muestra las instantáneas confirmadas. Te permite ver el historial del proyecto, filtrarlo y buscar cambios concretos. Mientras que git status te permite examinar el directorio de trabajo y el entorno de ensayo, git log solo muestra el historial confirmado.

La opción "log" se puede personalizar de diversas maneras, desde filtrar las confirmaciones hasta verlas en un formato completamente definido por el usuario. A continuación, presentamos algunas de las opciones de configuración más habituales de git log.

```
git log
```

Este comando muestra el historial de confirmaciones completo con el formato predeterminado. Si el resultado ocupa más de una pantalla, puedes usar Espacio para desplazarte y q para salir.

```
git log -n <limit>
```

Permite agrupar cada confirmación en una sola línea. Resulta útil para obtener información de alto nivel del historial del proyecto.

```
git log --oneline
git log --stat
```

Junto con la información habitual de git log, se incluye información sobre los archivos que se han modificado y el número relativo de líneas que se han añadido o eliminado en cada uno de ellos.

```
git log -p
```

Esta opción muestra el parche que representa cada confirmación. Se muestra la diferencia completa de cada confirmación, que es la vista más detallada que puedes tener del historial del proyecto.

```
git log --author="<pattern>"
```

Busca confirmaciones por un autor en concreto. El argumento puede ser una cadena sencilla o una expresión regular.

```
git log --grep="<pattern>"
```

Busca confirmaciones con un mensaje de confirmación que coincida con el patrón, que puede ser una cadena sencilla o una expresión regular.

```
git log <since>..<until>
```

Solo muestra las confirmaciones que se producen entre < since > y < until >. Estos argumentos pueden ser un ID de confirmación, un nombre de rama, HEAD o cualquier otro tipo de referencia de revisión.

```
git log <file>
```

Esta opción solo muestra las confirmaciones que incluyen el archivo especificado. Se trata de una forma sencilla de ver el historial de un archivo concreto.

```
git log --graph --decorate --oneline
```

A continuación, exponemos algunas opciones que debes tener en cuenta. La marca --graph dibujará un gráfico basado en texto de las confirmaciones de la parte izquierda de los mensajes de confirmación. --decorate añade los nombres de las ramas o las etiquetas de las confirmaciones que se muestran. --oneline muestra la información de confirmación en una única línea para que sea más fácil examinar las confirmaciones de un vistazo.

El comando git log es una herramienta básica de Git para explorar el historial del repositorio. Este comando se usa cuando necesitas buscar una versión concreta de un proyecto o saber los cambios que se introducirán mediante la fusión en una rama de función.

4.7 push/pull/fentch

El comando git push te permite subir los commits desde tu rama (branch) local en tu repositorio git local al repositorio remoto. Para poder subir a tu repositorio remoto, te debes asegurar de hacer commit a todos tus cambios al repositorio local.

La sintaxis de ese comando es la siguiente:

```
git push <nombre del repositorio> <nombre de la rama>
```

En orden para subir tu código, primero debes clonar un repositorio a tu máquina local.

```
# Una vez clonado el repositorio, estarás trabajando dentro de la rama por defecto (Por defecto es `main`)
git clone https://github.com/<git-usuario>/<nombre-repo> && cd <nombre-repo>
# Haz cambios y agrega tus archivos
git add <nombre-archivo>
# Ahora haz el commit de tu código
git commit -m ""“gregué cambios a mi repo!""# Sube los cambios en la rama ‘‘ain’’a github
```

```
git push origin main
```

Subir a un repositorio remoto específico y todas las ramas que contiene.

Si quieres subir todos tus cambios al repositorio remoto con todo y sus ramas, puedes utilizar:

```
git push --all <NOMBRE-REMOTO>
```

el cual:

> --all es la bandera que señala que quieres subir todas las ramas al repositorio remoto.
> NOMBRE-REMOTO es el nombre del repositorio remoto al cual las quieres subir.

Subir a una rama específica con el parámetro *force*

Si quieres ignorar los cambios locales hechos al repositorio Git en Github(Lo cual hacen la mayoría de desarrolladores para dar una solución rápida al servidor de desarrollo) luego puedes usar el comando —force para subir ignorando esos cambios.

```
git push --force <NOMBRE-REMOTO> <RAMA-REMOTA>
```

En la cual:

> NOMBRE-REMOTO es el nombre del repositorio remoto al cual quieres subir los cambios.
> RAMA-REMOTA es el nombre de la rama remota a la cual quieres subir los cambios.

Subir ignorando el hook pre-push de Git

Por defecto git push disparará el interruptor --verify. Esto significa que git ejecutará cualquier script pre-push(antes de subirlo) del lado del cliente, que haya sido configurado. Si el script pre-push falla, también fallará el git push. (Hooks Pre-Push son buenos para hacer cosas como, revisar si los mensajes al hacer commit están dentro del estándar de la empresa, ejecutar tests unitarios, etc...). Ocasionalmente podrías querer ignorar este comportamiento por defecto, por ejemplo: en el escenario donde quisieras subir tus cambios a una rama en particular para que otro colaborador la descargue, pero los cambios en tu trabajo en progreso rompen los tests unitarios. Para ignorar el hook, simplemente inserta tu comando push y añade la bandera --no-verify

```
git push --no-verify
```

git pull es un comando de Git utilizado para actualizar la versión local de un repositorio desde otro remoto. Es uno de los cuatro comandos que solicita interacción de red por Git. Por default, git pull hace dos cosas:

- Actualiza la rama de trabajo actual (la rama a la que se ha cambiado actualmente)
- Actualiza las referencias de rama remota para todas las demás ramas.

git pull recupera (git fetch) las nuevas confirmaciones y las fusiona (git merge) en tu rama local. La síntaxis de este comando es el siguiente:

```
# Formato General
git pull OPCIONES REPOSITORIO REFSPEC

# Pull de una rama específica
git pull NOMBRE-REMOTO NOMBRE-RAMA
```

En donde:

- OPCIONES son las opciones de comandos, como --quiet o --verbose.
- REPOSITORIO es la URL de tu repositorio
- REFSPEC especifica cuáles referencias recuperar y cuáles referencias locales actualizar.
- NOMBRE-REMOTO es el nombre de tu repositorio remoto.
- NOMBRE-RAMA es el nombre de tu rama.

Nota

Si tienes cambios no confirmados, la parte de fusión del comando git pull fallará y tu rama local quedará intacta. Por lo tanto, siempre deberías confirmar (commit) tus cambios en una rama antes de actualizar nuevas confirmaciones de un repositorio remoto.

Nos queda ver que un pull, git pull, es un comando combinado, que equivale a git fetch + git merge.

git fetch

Por sí mismo, git fetch actualiza todas las referencias de rama remota en tu repositorio local. En realidad, no se reflejan cambios en ninguna de las ramas de trabajo locales.

git merge

Sin ningún argumento, git merge fusionará la referencia de rama remota correspondiente a la rama de trabajo local.

5.- Mecanismos de actualización

Una de las tareas más importantes que se hacen en Git es la confirmación de cambios en repositorios, lo que se llama comúnmente un "commit". Mediante esta operación se envían los cambios al control de versiones, creando un nuevo estado del proyecto, que será almacenado mediante Git.

El proceso de una confirmación es el siguiente:

- El desarrollador hace cambios en el código del proyecto
- Los cambios se envían al staging area de Git, con el comando "add"
- Los cambios se confirman con el comando "commit" y pasan a formar parte del histórico de confirmaciones del repositorio

Una vez se han confirmado cambios en Git podremos acceder al estado del proyecto, tal como estaba en esa confirmación. Además, en todo momento podremos ver qué cambios se hicieron en el código del proyecto entre esa confirmación y la anterior.

5.1 add

El comando git add añade un cambio del directorio de trabajo en el entorno de ensayo. De este modo, indica a Git que quieres incluir actualizaciones en un archivo concreto en la próxima confirmación. Sin embargo, git add no afecta al repositorio de manera significativa: en realidad, los cambios no se registran hasta que ejecutas git commit. Junto con estos comandos, también necesitarás git status para ver el estado del directorio de trabajo y el entorno de ensayo.

Los comandos git add y git commit componen el flujo de trabajo fundamental de Git. Estos son los dos comandos que todos los usuarios de Git deben conocer, con independencia del modelo de colaboración de su equipo. Constituyen el medio para registrar las versiones de un proyecto en el historial del repositorio.

El desarrollo de un proyecto gira en torno al patrón básico de edición/preparación/confirmación. Primero, editas tus archivos en el directorio de trabajo. Cuando lo tienes todo listo para guardar una copia del estado del proyecto, preparas los cambios con git add. Cuando ya tienes la instantánea preparada a tu gusto, la confirmas en el historial del proyecto con git commit. El comando git reset sirve para deshacer una confirmación o una instantánea preparada.

Además de git add y git commit, hay un tercer comando (git push) que resulta esencial para lograr un flujo de trabajo colaborativo completo de Git. Esto permite a otros miembros del equipo acceder a un conjunto de cambios guardados.

El comando git add no debe confundirse con svn add, que añade un archivo al repositorio. En su lugar, git add actúa en el nivel más abstracto de los cambios. Esto significa que git add debe aplicarse cada vez que se modifica un archivo, mientras que svn add solo tiene que aplicarse una vez por cada archivo. Puede parecer redundante, pero este flujo de trabajo facilita muchísimo la organización de los proyectos.

```
git add <file>
```

Prepara todos los cambios de <file> para la siguiente confirmación.

```
git add <directory>
```

Prepara todos los cambios de <directory> para la siguiente confirmación.

```
git add -p
```

Inicia una sesión de entorno de ensayo interactiva que te permite elegir las partes de un archivo que quieres añadir a la siguiente confirmación. Se te presentarán una serie de cambios y se te solicitará un comando. Utiliza y para preparar el fragmento, n para ignorarlo, s para dividirlo en fragmentos más pequeños, e para editarlo manualmente y q para salir.

5.2 commit

El comando git commit captura una instantánea de los cambios preparados en ese momento del proyecto. Las instantáneas confirmadas pueden considerarse como versiones "seguras" de un proyecto: Git no las cambiará nunca a no ser que se lo pidas expresamente. Antes de ejecutar git commit, se utiliza el comando git add para pasar o "preparar" los cambios en el proyecto que se almacenarán en una confirmación. Estos dos comandos, git commit y git add, son dos de los que se utilizan más frecuentemente.

```
git commit
```

Confirma la instantánea preparada. El comando abrirá un editor de texto que te pedirá un mensaje para la confirmación. Una vez escrito el mensaje, guarda el archivo y cierra el editor para crear la confirmación.

```
git commit -a
```

Confirma una instantánea de todos los cambios del directorio de trabajo. Esta acción solo incluye las modificaciones a los archivos con seguimiento (los que se han añadido con git add en algún punto de su historial).

```
git commit -m "commit message"
```

Un comando de atajo que crea inmediatamente una confirmación con un mensaje de confirmación usado. De manera predeterminada, git commit abrirá el editor de texto configurado localmente y solicitará que se introduzca un mensaje de confirmación. Si se usa la opción -m, se omitirá la solicitud de editor de texto a favor de un mensaje insertado.

```
git commit -am "commit message"
```

Un comando de atajo para usuarios avanzados que combina las opciones -a y -m. Esta combinación crea inmediatamente una confirmación de todos los cambios preparados y aplica un mensaje de confirmación insertado.

```
git commit –amend
```

Esta opción añade otro nivel de funcionalidad al comando confirmado. Al pasar esta opción, se modificará la última confirmación. En vez de crear una nueva confirmación, los cambios preparados se añadirán a la confirmación anterior. Este comando abrirá el editor de texto configurado del sistema y te pedirá que cambies el mensaje de confirmación especificado anteriormente.

5.3 merge

El comando git merge fusionará cualquier cambio que se haya hecho en la base de código en una rama separada de tu rama actual como un nuevo commit. La sintaxis del comando es la siguiente:

```
git merge NOMBRE-DE-LA-RAMA
```

Por ejemplo, si estás trabajando actualmente en una rama llamada dev y deseas fusionar los nuevos cambios que se hayan realizado en una rama llamada new-features, ejecutarías el siguiente comando:

```
git merge new-features
```

Nota**:** Si hay algún cambio al que no se le ha hecho commit en la rama actual, Git no te permitirá fusionar hasta que se hayan realizado commit todos los cambios en tu rama actual.

Crear una nueva rama y realizar commit a los cambios

```
git checkout -b nombre-de-la-nueva-rama
git add .
git commit -m "<tu mensaje de commit>"
```

Guardarlos en el stash (almacenar temporalmente los cambios que hayas efectuado en el código en el que estás trabajando para que puedas trabajar en otra cosa y, más tarde, regresar y volver a aplicar los cambios más tarde.)

```
git stash           # agregarlos al stash
git merge new-features  # fusionarlos
git stash pop       # obtén los cambios devuelta al árbol de trabajo (working tree)
```

Abandonar todos los cambios

```
git reset --hard      # remueve todos los cambios pendientes
```

5.4 Git Rebase

Realizar un rebase a una rama (branch) en Git es una forma de mover la totalidad de una rama a otro punto del árbol. El ejemplo más simple es mover una rama más arriba en el árbol. Digamos que tenemos una rama que se separó de la rama master en el punto A:

```
          /o-----o---o--o-----o--------- branch
--o-o--A--o---o---o---o----o--o-o-o--- master
```

Cuando se realiza rebase se puede mover así:

```
                                        /o-----o---o--o-----o------ branch
--o-o--A--o---o---o---o----o--o-o-o master
```

Para realizar rebase, asegúrate de tener todos los commits que quieras en el rebase en tu rama master. Revisar la rama en la que quieres hacer el rebase y escribe git rebase master (donde master es la rama en la que quieres hacer el rebase).

También es posible hacer rebase en una rama diferente, de modo que, por ejemplo, una rama que se basaba en otra rama (llamémosla feature) se rebasa en master:

```
                                    /---o-o branch
              /---o-o-o-o---o--o------ feature
```

----o--o-o-A----o---o--o-o-o--o--o- master

Después de git rebase master branch o git rebase master si te encuentras (checked out) en branch, obtendrás:

```
                /---o-o-o-o---o--o------ feature
----o--o-o-A----o---o--o-o-o--o--o- master
                                  \---o-o Branch
```

5.5 Merge Conflicts

Un merge conflict es cuando realizar commits en ramas separadas que alteran la misma línea de manera conflictiva. Si esto ocurre, Git no sabrá que versión del archivo guardar en un mensaje de error similar al siguiente:

```
CONFLICT (content): Merge conflict in resumé.txt Automatic merge failed; fix conflicts and then commit the result.
```

Si miras el archivo resumé.txt en tu editor de texto, puedes ver dónde tuvo lugar el conflicto:

```
<<<<<<< HEAD
Address: 808 South Street
=======
Address: 505 North Street
>>>>>>> updated_address
```

Git agregó algunas líneas adicionales en el archivo:

- <<<<<<< HEAD
- =======
- >>>>>>> updated_address

Piensa en ======= como la línea divisoria del conflicto. Todo lo que hay entre <<<<<<< HEAD y ======= es el contenido de la rama actual a la que apunta HEAD ref. Por otro lado, todo lo que hay entre ======= y >>>>>>> updated_address es el contenido de la rama que se está fusionando, updated_address.

5.6 Git Merge vs Git Rebase

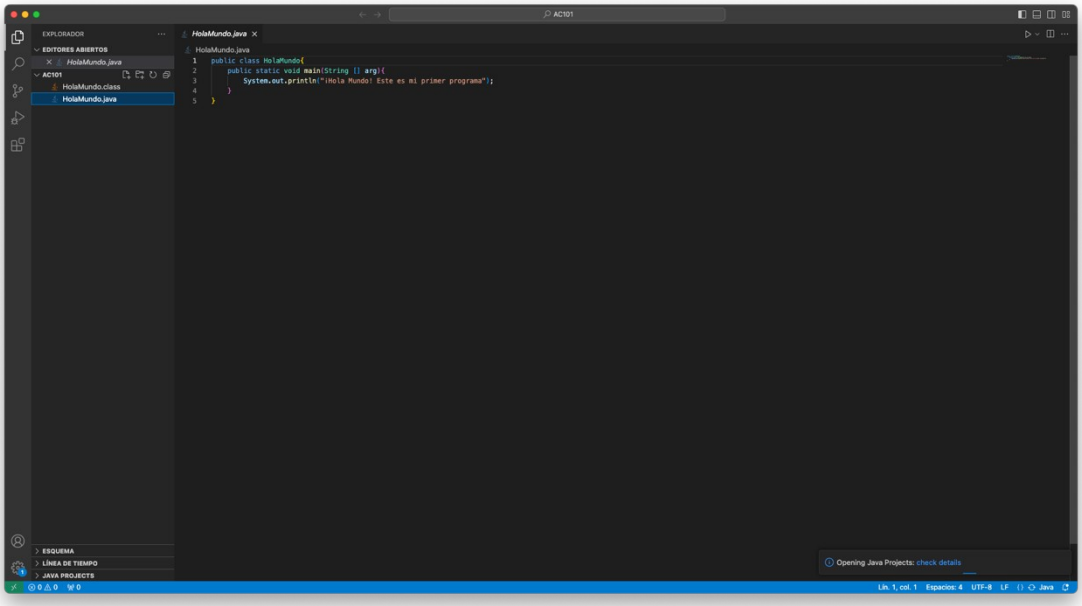
Tanto git merge como git rebase son comandos muy útiles, y uno no es mejor que el otro. Sin embargo, hay algunas diferencias muy importantes entre los dos comandos que tú y tu equipo deben tener en cuenta.

Cada vez que se ejecuta git merge, se crea un merge commit extra. Siempre que trabajes en tu repositorio local, tener demasiados merge commits puede hacer que la historia de commits parezca confuso. Una forma de evitar el merge commit es usar git rebase en su lugar.

git rebase es una característica muy poderosa. Dicho esto, también es arriesgado si no se usa de la manera correcta. git rebase altera el historial de commits, así que úsalo con cuidado. Si el rebase se hace en el repositorio remoto, entonces puede crear muchos problemas cuando otros desarrolladores intentan sacar los últimos cambios de código del repositorio remoto. Recuerda que sólo debes ejecutar git rebase en un repositorio local.

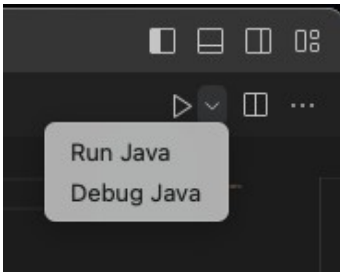
6.- Uso básico de un entorno de desarrollo: Edición de programas. Generación de ejecutables.

El uso de IDES es una de las cosas que ha cambiado el hecho de programar. Esto es debido a que nos facilitan tanto la creación como la edición de programas. Basta con abrir el proyecto o fichero y ponernos a escribir o modificar el código que consideremos necesario.



Pero donde tenemos una gran ventaja es a la hora de generar los ficheros ejecutables con el IDE.

El IDE nos permite generar los ejecutables sin la necesidad de hacerlo por terminal. De ese modo, en VS Code, podemos darle al botón de “Run Java” y nos generará el ejecutable de nuestro programa.



Esto es un gran avance al no tener que pasar por terminal y tener que hacer uso de javac y de la problemática que nos puede dar, el ya lo hace por nosotros y en caso de error, nos lo mostrará.

7.- Ventajas e inconvenientes del desarrollo de aplicaciones con entornos de desarrollo con respecto al uso de tecnologías tradicionales.

A la hora de trabajar con IDE tenemos que ser capaces de reconocer las ventajas de su uso. Está claro que se puede utilizar cualquier editor de texto para escribir código. Sin embargo, la mayoría de los entornos de desarrollo integrados incluyen funcionalidades que van más allá de la edición de texto. Proporcionan una interfaz central para herramientas de desarrollo comunes, lo que hace que el proceso de desarrollo de software sea mucho más eficiente. Los desarrolladores pueden comenzar a programar aplicaciones nuevas rápidamente en lugar de integrar y configurar diferentes software de forma manual. También prescinden de la necesidad de conocer todas las herramientas y, en su lugar, pueden centrarse en una única aplicación. A continuación, se describen algunas razones por las que los desarrolladores utilizan los IDE:

Automatización de la edición de código

Los lenguajes de programación tienen reglas sobre cómo estructurar instrucciones. Dado que un IDE conoce estas reglas, contiene muchas funciones inteligentes para escribir o editar automáticamente el código fuente.

Resaltado de sintaxis

Un IDE puede dar formato al texto escrito haciendo que algunas palabras aparezcan en negritas o itálicas, o utilizando diferentes colores de fuente. Estas pistas visuales hacen que el código fuente sea más legible y dan retroalimentación instantánea sobre errores sintácticos accidentales.

Finalización de código inteligente

Varios términos de búsqueda aparecen cuando comienza a escribir palabras en un motor de búsqueda. De manera similar, un IDE puede proponer sugerencias para completar una instrucción de código cuando el desarrollador comienza a escribir.

Refactorización del soporte

La refactorización de código es el proceso de reestructuración del código fuente para hacerlo más eficiente y legible sin tener que cambiar su funcionalidad central. Los IDE pueden autorefactorizarse hasta cierto punto, lo que permite a los desarrolladores mejorar su código de forma fácil y rápida. Otros miembros del equipo entienden el código legible con mayor rapidez, lo que respalda la colaboración dentro del equipo.

Automatización de la creación local

Los IDE aumentan la productividad de los programadores, pues realizan tareas de desarrollo recurrentes que por lo general son parte de todos los cambios de código. Estos son algunos ejemplos de tareas de codificación regulares que lleva a cabo un IDE.

Compilación

Un IDE compila o convierte el código en un lenguaje simplificado que el sistema operativo puede entender. Algunos lenguajes de programación implementan la compilación “justo a tiempo”, donde el IDE convierte código legible para los humanos en código para máquinas desde el interior de la aplicación.

Pruebas

El IDE permite a los desarrolladores automatizar pruebas de unidades localmente antes de integrar el software con el código de otros desarrolladores; también se llevan a cabo pruebas de integración más complejas.

Depuración

La depuración es el proceso de corregir todos los errores o fallas revelados en las pruebas. Uno de los aspectos más valiosos e importantes de un IDE para fines de depuración es el hecho de poder seguir el código, línea por línea, conforme se pone en marcha e inspeccionar el comportamiento del código. Los IDE también integran varias herramientas de depuración que destacan las fallas provocadas por errores humanos en tiempo real, incluso mientras el desarrollador está escribiendo.

El mayor inconveniente que presenta un IDE es el consumo de recursos. Al ser un entorno de desarrollo, consume muchos más recursos que un simple editor de texto. A esto se le puede sumar el hecho que el usuario deja de ser consciente de los pasos o acciones que se han de dar en el desarrollo de

software, pero eso es un debate que no pertenece a este tema o a este módulo.

Es por ello que la aparición de los IDE fue un gran hito en la industria informática al facilitar mucho las fases de desarrollo y la vida al programador.